SDU
University of Southern Denmark

Brief Announcement

Distributed Graph Algorithms with Predictions

Boyar, Joan; Ellen, Faith; Larsen, Kim S.

Published in:
Proceedings of the ACM Symposium on Principles of Distributed Computing

DOI:
10.1145/3732772.3733530

Publication date:
2025

Document version:
Final published version

Document license:
CC BY

Citation for pulished version (APA):
Boyar, J., Ellen, F., & Larsen, K. S. (2025). Brief Announcement: Distributed Graph Algorithms with Predictions. In *Proceedings of the ACM Symposium on Principles of Distributed Computing* (pp. 322-325). Association for Computing Machinery. https://doi.org/10.1145/3732772.3733530

Go to publication entry in University of Southern Denmark's Research Portal

Terms of use
This work is brought to you by the University of Southern Denmark.
Unless otherwise specified it has been shared according to the terms for self-archiving.
If no other license is stated, these terms apply:

- You may download this work for personal use only.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying this open access version

If you believe that this document breaches copyright please contact us providing details and we will investigate your claim.
Please direct all enquiries to puresupport@bib.sdu.dk

Download date: 12. Oct. 2026



Brief Announcement: Distributed Graph Algorithms with Predictions

Joan Boyar
joan@imada.sdu.dk
University of Southern Denmark
Odense, Denmark

Faith Ellen
faith@cs.toronto.edu
University of Toronto
Toronto, Canada

Kim S. Larsen
kslarsen@imada.sdu.dk
University of Southern Denmark
Odense, Denmark

Abstract

We initiate the study of distributed graph algorithms with predictions in synchronous message passing systems. Each node in the graph is given a prediction, which is some extra information about the problem instance that may be incorrect. The better the prediction, the fewer rounds the algorithm should perform. We present a framework for evaluating distributed graph algorithms with predictions and some methods for transforming existing algorithms without predictions to effectively use predictions. Our approach is illustrated using the Maximal Independent Set problem.

CCS Concepts

• **Theory of computation** → **Distributed algorithms**; *Graph algorithms analysis*; *Algorithm design techniques*; • **Mathematics of computing** → **Graph algorithms**.

Keywords

algorithms with predictions, synchronous message passing, maximal independent set

ACM Reference Format:
Joan Boyar, Faith Ellen, and Kim S. Larsen. 2025. Brief Announcement: Distributed Graph Algorithms with Predictions. In *ACM Symposium on Principles of Distributed Computing (PODC '25), June 16–20, 2025, Huatulco, Mexico.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3732772.3733530

1 Introduction

The study of algorithms with predictions is a very active research area [1]. An algorithm with predictions receives the input to a problem instance plus additional information that may come from machine learning or some other source that is treated as a black box. It may help the algorithm solve the instance by compensating for a lack of knowledge about global information or by providing the solution to a related instance, such as a graph with the same set of nodes, but where some of the edges may have changed. Different predictions may be appropriate for different problems. One may also consider different predictions for the same problem.

Predictions may have errors. An *error measure* is a function, η, that takes a problem instance and a prediction and produces a number that describes the amount of error in the prediction. It is part of the definition of a problem with predictions. An algorithm with predictions should perform very well when the predictions are good and not perform significantly worse than a good algorithm without predictions. The degradation in performance should not grow too fast as the error measure grows.

Our model of computation is a synchronous message passing system, such as LOCAL, where the processes are the nodes in a graph, each of which has a different identifier drawn from $\{1, \ldots, d\}$. Nodes are assumed to be non-faulty and know d, the identifiers of their neighbors, the number of nodes, n, and possibly an upper bound, Δ, on the maximum degree of the nodes. We measure the performance of an algorithm by the maximum number of rounds until all nodes have terminated, as a function of these parameters.

In this paper, the prediction for an instance consists of a potential output for each node. For example, in the Maximal Independent Set (MIS) problem, each node, p_i, must output a bit, y_i, so that the set of nodes that output 1 form a maximal independent set of the graph. A prediction for this problem consists of a bit, x_i, given to each node, p_i. The set of nodes that are given 1 is predicted to be a maximal independent set of the graph. There are two possible types of errors that can occur in a prediction for this problem: two neighboring nodes could both receive 1, so the prediction does not describe an independent set, or a node and all of its neighbors could receive 0, so the prediction is not maximal.

2 Framework

Inspired by definitions from online algorithms with predictions, we propose the following definitions to describe the performance of distributed graph algorithms with predictions with respect to an error measure η. Consider an algorithm, A, that solves a graph problem despite possible errors in the predictions and an algorithm B without predictions that solves the same problem.

- A has *consistency* $c(n)$ if A terminates in $c(n)$ rounds when the prediction is correct (i.e., $\eta = 0$). A is *consistent* if $c(n)$ is at most a constant times the optimal number of rounds needed to check whether a predicted solution is correct.
- A is $f(\eta)$*-degrading* if it terminates within $f(\eta)+c(n)+O(1)$ rounds.
- A is *robust with respect to* B if it terminates within $O(g(n))$ rounds, where $g(n)$ is the round complexity of B.
- A is *robust* if it is robust with respect to an asymptotically optimal algorithm for the problem.
- A is *smooth with respect to* B if it is consistent, robust with respect to B, and $f(\eta)$-degrading, where f is a function that does not grow too quickly, for example, linear or polynomial in η.



This work is licensed under a Creative Commons Attribution International 4.0 License.
PODC '25, Huatulco, Mexico
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1885-4/25/06
https://doi.org/10.1145/3732772.3733530

- A is *smooth* if it is consistent, robust, and $f(\eta)$-degrading, where f is a function that does not grow too quickly.

Sometimes, Δ is included as a parameter of the functions c, f, and g. Note that consistency and robustness are standard terms used in the online algorithms with predictions literature, so we use them here, even though they have different meanings in the theory of distributed computing. In other contexts, degradation is seldom used and, like here, smoothness is only defined informally.

3 Extendable Partial Solutions, Initialization Algorithms, and Error Measures

The outputs of a subset of nodes is a *partial solution* to a graph problem if they are a solution to the problem on the subgraph induced by these nodes. If a partial solution, together with any solution to the problem on the subgraph induced by the other nodes, is a solution to the problem on the entire graph, we say that this partial solution is *extendable*. In particular, a solution to the graph problem is an extendable partial solution. A *greedily completable problem* [3] is a problem such that any partial solution can be extended to a complete solution in a greedy manner, in an arbitrary order. Some graph problems, such as Maximal Independent Set, are not greedily completable, but they have the property that certain partial solutions are extendable. For example, if I is any independent set of a graph, then the partial solution in which all nodes in I output 1 and all their neighbors output 0 is an extendable partial solution.

To ensure consistency, a distributed graph algorithm with predictions has to begin by determining whether the predictions given to the nodes describe a correct solution to the problem instance. If so, the nodes output this solution and terminate. Otherwise, some (possibly empty) subset of nodes should output an extendable partial solution. The remaining nodes stay active and can proceed to compute a solution on the remaining graph. We call this an *initialization algorithm*. For example, there is a simple 3 round initialization algorithm for the Maximal Independent Set problem: In round 1, nodes exchange their predictions with their neighbors. The nodes that have prediction 1 and whose neighbors have prediction 0 form an independent set I. In round 2, every node in I notifies its neighbors, outputs 1, and terminates. In round 3, every node that is a neighbor of a node in I notifies its neighbors, outputs 0, and terminates. This initialization algorithm has the additional property that the extendable partial solution that is output at nodes agrees with the prediction at these nodes. Such an algorithm is called a *pruning algorithm* [8].

More generally, an initialization algorithm is allowed to change the prediction at some nodes. For example, suppose the set I in the algorithm above is defined, instead, to be the set of all nodes with prediction 1 whose neighbors with prediction 1 (if any) all have smaller identifiers. Then a possibly larger extendable partial solution is obtained. Other graph problems with similar pruning and initialization algorithms include $(\Delta + 1)$-Vertex Coloring, $(2\Delta - 1)$-Edge Coloring, and Maximal Matching.

For each graph problem, fix a pruning algorithm. If the extendable partial solution output by this algorithm is not a solution to the problem instance, then the remainder of the graph consists of one or more components, called the *error components*. Because this partial solution is extendable, it would suffice to compute a solution on each component separately. Hence, we use the maximum of the number of nodes in each error component as our error measure, η. Note that the value of the error measure on a problem instance is not known by the nodes.

Even if a distributed graph algorithm with predictions does not start with this pruning algorithm, the performance of the algorithm on each problem instance will still be evaluated as a function of η. But some (possibly different) initialization algorithm should be used at the beginning of the algorithm, to ensure consistency.

In the full version of the paper [6], we consider other error measures, including the maximum of the diameter of each error component. Some seemingly natural error measures are not good, for example, the minimum number of predictions that would have to be changed to obtain a correct solution. Another is the number of nodes for which the predictions are not a partial solution to the problem on the subgraph induced by the node and its neighbors. One difficulty is that these error measures do not reflect the fact that nodes in different error components can work independently to find outputs that result in a solution for the entire problem.

4 Component-Size Bounded Algorithms

Consider a graph algorithm (without predictions) whose round complexity depends only on the size of the graph, *without* the nodes knowing that size, or even an upper bound on the size. In particular, it cannot depend on the size of the domain from which the nodes are given identifiers or the largest node identifier. Such an algorithm can be used as part of a graph algorithm with predictions to separately compute a solution on each error component. We call such an algorithm a *component-size bounded algorithm*.

A graph algorithm is *uniform* with respect to a graph parameter [8] if the round complexity of the algorithm depends on the value of that parameter, but the algorithm does not require that nodes know even an upper bound on the value of the parameter. Since an algorithm can be uniform with respect to more than one parameter, component-size bounded algorithms are a special case of algorithms that are uniform with respect to size.

An example of a component-size bounded algorithm for the Maximal Independent Set problem is to repeatedly choose an independent set of nodes in the remaining graph, add these nodes to the independent set, and remove them and their neighbors from the graph. One way to deterministically choose an independent set is to take every node whose identifier is larger than the identifiers of all its active neighbors. We call this the *Distributed Greedy MIS Algorithm*. During each odd numbered round, at least one active node in each component of the remaining graph has an identifier that is larger than all its neighbors and, hence, terminates. If a component contains more than one node, then at least one active node in the component receives a message from a neighbor and, hence, terminates during the subsequent even numbered round. Thus, the Distributed Greedy MIS Algorithm has round complexity at most n when performed on a graph with n nodes. In the full version of the paper, we prove (using Ramsey theory) that it is an asymptotically optimal deterministic component-size bounded algorithm when identifiers can be sufficiently large.

Theorem 4.1. *Every deterministic component-size bounded algorithm for computing a maximal independent set of a line with n nodes requires at least* $(n-5)/2$ *rounds in the LOCAL model.*

Korman, Sereni and Viennot [8] give a method for constructing algorithms that are uniform with respect to n and Δ from ones that are not, without increasing the running time by more than a constant factor. Their idea is to repeatedly execute the original algorithm with increasingly larger guesses for n (and Δ), but store the values of the output variables as a tentative solution, rather than outputting them. After each repetition, they apply a pruning algorithm, output the extendable partial solution it produces, and perform the next repetition on the remainder of the graph. However, algorithms depending on the nodes having distinct identifiers and whose round complexity depends on d remain nonuniform with respect to d. In particular, they obtain a $(\Delta+1)$-Vertex Coloring algorithm running in $O(\Delta + \log^* d)$ rounds that is uniform with respect to Δ.

5 Methods for Constructing Graph Algorithms with Predictions

One way to obtain a graph algorithm with predictions is to run an initialization algorithm and then run a component-size bounded algorithm independently on each error component. For example, running an initialization algorithm for the Maximal Independent Set problem followed by the Distributed Greedy MIS Algorithm (on each error component) is a distributed Maximal Independent Set algorithm with predictions that is consistent and has round complexity $\eta+3$. Thus, it is η-degrading. Alternatively, after initialization, one could run the Δ-uniform algorithm for $(\Delta+1)$-Vertex Coloring in [8] and then use a greedy algorithm to obtain a maximal independent set from the coloring. Specifically, the greedy algorithm iterates through the colors in increasing order: All active nodes of that color notify their neighbors, output 1, and terminate. Then all active neighbors of these nodes output 0 and terminate. The resulting algorithm with predictions is consistent and has round complexity $O(\Delta' + \log^* d)$, where Δ' is the maximum of the degrees of the error components. Since the degree of an error component is less than the number of nodes in that error component, the round complexity is $O(\eta + \log^* d)$. Hence, it is almost $O(\eta)$-degrading. Note that any deterministic algorithm for the Maximal Independent Set problem on graphs with n nodes and degree Δ requires $\Omega(\min\{\Delta+\log^* n, (\log n)/\log\log n\})$ rounds in the LOCAL model [2]. When $d \in n^{O(1)}$ and $\Delta \in O((\log n)/\log\log n)$, this algorithm with predictions is robust.

A *clean-up* algorithm extends a partial solution by having a (possibly empty) set of nodes output values, so that the resulting partial solution is extendable. Provided that nodes inform their active neighbors of their outputs prior to terminating, a clean-up algorithm is similar to an initialization algorithm for the same problem. For the Maximal Independent Set problem, a clean-up algorithm can be performed in one round: every active node which has a neighbor that output 1 simply outputs 0 after informing its active neighbors that it will output 0.

Suppose there is an algorithm, B, without predictions whose round complexity depends on graph parameters that are known to all nodes. Using a component-size bounded algorithm, D, with $f(n)$ round complexity, we can obtain an algorithm with predictions that is $2f(\eta)$-degrading and is robust with respect to B. The idea is to run an initialization algorithm, then run D for r rounds, next run a clean-up algorithm, C, and finally run B on the remaining graph. The number of rounds, r, performed by D is the sum of the round complexities of algorithms B and C. If $f(\eta) \le r$, then at most $f(\eta)$ rounds are performed after initialization; otherwise, at most $2r < 2f(\eta)$ rounds are performed.

For example, consider the Maximal Independent Set algorithm by Ghaffari and Grunau [7], with $\tilde{O}(\log^{5/3} n)$ round complexity. It depends on nodes knowing an upper bound $N \in n^{O(1)}$ on the number of nodes in the graph and assumes that $d \in n^{O(1)}$. A distributed graph algorithm with predictions for the Maximal Independent Set problem that is consistent, robust with respect to this algorithm, and 2η-degrading can be obtained using their algorithm as algorithm B and the Distributed Greedy MIS Algorithm as the component-size bounded algorithm, D.

The solutions to some graph problems remain solutions when nodes are removed from the graph. For example, a coloring of a graph is still a coloring of any induced subgraph. An algorithm for such a problem is *fault-tolerant* if it runs in a model where nodes can crash and, at any point, the partial solution output by the nodes that have not crashed is a partial solution to the problem on the subgraph induced by these nodes.

Suppose that algorithm B has two phases, B' and B'', where B' is fault-tolerant and both B'' and C have constant round complexity. Then we can obtain an algorithm with predictions that is $f(\eta)$-degrading and is robust with respect to B, as follows: First run an initialization algorithm. Then run D in parallel with B' (instead of prior to B'). To avoid interference between the two algorithms, nothing is output during B'. Instead, any outputs computed during B' are stored locally by the nodes. A node that outputs a value and terminates while it is performing D is treated by algorithm B' as if it has crashed (even though the model assumes that nodes are reliable). After B' is known to have completed, the partial solution produced by D is made extendable by C. Then the locally stored outputs, if any, computed by B' are output. Finally, B'' is run.

For example, there is a $(\Delta+1)$-Vertex Coloring algorithm [4, 5, 9] that is fault-tolerant and has round complexity $O(\Delta+\log^* d)$, but is not uniform with respect to Δ. Let B' be this algorithm and let B'' be the greedy algorithm that produces a maximal independent set from the coloring in Δ rounds. When Δ is known to be constant, this method results in a distributed graph algorithm with predictions for the Maximal Independent Set problem that is consistent, robust, and η-degrading. Hence, it is smooth.

Acknowledgments

We would like to thank Jukka Suomela for helpful discussion, including suggesting Theorem 4.1 and its proof. Joan Boyar and Kim S. Larsen were supported in part by the Independent Research Fund Denmark, Natural Sciences, grants DFF-0135-00018B and DFF-4283-00079B. Faith Ellen was supported in part by the Natural Science and Engineering Research Council of Canada (NSERC) grant RGPIN-2020-04178.

References

[1] Algorithms with Predictions. 2024. Retrieved April 18, 2025 from https://algorithms-with-predictions.github.io/

[2] Alkida Balliu, Sebastian Brandt, Juho Hirvonen, Dennis Olivetti, Mikaël Rabie, and Jukka Suomela. 2021. Lower Bounds for Maximal Matchings and Maximal Independent Sets. *Journal of the ACM* 68, 5 (2021), 39:1–39:30. doi:10.1145/3461458

[3] Alkida Balliu, Juho Hirvonen, Darya Melnyk, Dennis Olivetti, Joel Rybicki, and Jukka Suomela. 2022. Local Mending. In *29th International Colloquium on Structural Information and Communication Complexity (SIROCCO) (Lecture Notes in Computer Science, Vol. 13298)*. Springer, 1–20. doi:10.1007/978-3-031-09993-9_1

[4] Leonid Barenboim and Michael Elkin. 2013. *Distributed Graph Coloring: Fundamentals and Recent Developments*. Morgan & Claypool Publishers. doi:10.2200/S00520ED1V01Y201307DCT011

[5] Leonid Barenboim, Michael Elkin, and Uri Goldenberg. 2022. Locally-iterative Distributed $(\Delta + 1)$-coloring and Applications. *Journal of the ACM* 69, 1 (2022), 5:1–5:26. doi:10.1145/3486625

[6] Joan Boyar, Faith Ellen, and Kim S. Larsen. 2025. Distributed Graph Algorithms with Predictions. arXiv:2501.05267 [cs.DC]. doi:10.48550/arXiv.2501.05267

[7] Mohsen Ghaffari and Christoph Grunau. 2024. Near-Optimal Deterministic Network Decomposition and Ruling Set, and Improved MIS. In *65th IEEE Annual Symposium on Foundations of Computer Science (FOCS)*. 2148–2179. doi:10.1109/FOCS61266.2024.00007

[8] Amos Korman, Jean-Sébastien Sereni, and Laurant Viennot. 2013. Toward more localized local algorithms: removing assumptions concerning global knowledge. *Distributed Computing* 26, 5-6 (2013), 289–308. doi:10.1007/S00446-012-0174-8

[9] Nathan Linial. 1992. Locality in Distributed Graph Algorithms. *SIAM J. Comput.* 21, 1 (1992), 193–201. doi:10.1137/0221015